REPORT · 2026 年 10 月 26 日

中小型文物三维建模自动化系统 v1.1.9-r13 发行说明

交付一致性 · 纹理质检 · 五态终态

H3D 项目组
2026 年 10 月 26 日

目 录

一、发行概览 3

二、本版改动总表 5

三、缺陷①：2048² 图集没进交付包 6

3.1 现象：264 万纹素，只交出 8.5 万 . . . 6

3.2 根因：图集的落盘时机被绑在“导出哪些格式”上 . . . 6

3.3 修复：先把图集写下来，再单独判断有没有模型引用它 . . . 6

3.4 交付层：图集与模型格式无关，一律进包 . . . 7

四、缺陷②：M5 纹理质检整段失效 8

4.1 现象：五项 N/A，与 M4 的 94.89% 自相矛盾 . . . 8

4.2 根因：与缺陷①同一个 . . . 8

4.3 修复：三级定位 + 成因分类 . . . 8

4.4 实测结果：五项指标回来了，且与 M4 对得上 . . . 8

五、缺陷③：终态又归并了 10

5.1 现象：「完成 7 · 未执行 1 → 部分完成」 . . . 10

5.2 根因：配置关闭的阶段根本不进 manifest . . . 10

5.3 修复：阶段终态扩成五态 . . . 10

5.4 报表：分母只算已启用阶段 . . . 11

六、缺陷④：元数据与产物对不上 12

6.1 三处不一致 . . . 12

6.2 修复 . . . 12

七、验证 13

7.1 单元测试 . . . 13

7.2 真机复跑（同一 30 张照片项目） . . . 13

八、已知限制与下一步 14

一、发行概览

本次发行版本 **v1.1.9（修订 r13）**，打包日期 2026-10-26。触发这一版的是一次**非常成功**的实机运行：30 张照片、纯照片场景，31.4 分钟出模，几何 59,094 顶点 / 117,662 面、纹理岛内填充率 94.89%、相对中误差 0.70%。也就是说**算法链本身没有问题**。问题出在它之后——算出来的东西没有被交出去，没做的事被显示成没事。

三件事的共同点是一句话：**管线算对了，但交付与报表没跟上。**① M4 花了 50.18 秒逐纹素采样出 2048² 图集（2,639,926 个纹素），交付包里却只有 85,420 个顶点色，差 31 倍；② M5 的纹理质检整段失效，五项指标全 N/A，而同一份报表里写着填充率 94.89%；③ 终态又把「配置关闭」和「上游失败」归成一类，印出「完成 7 · 未执行 1 → 部分完成」。

> **一句话结论**：本版让**交付包里的模型真正带得动那张图集**——即使配置写的是 export_formats: [ply]（PLY 无法引用图像纹理），图集也会无条件落盘、进交付包，并在没有任何格式能承载它时自动补导 OBJ。实测交付包从「1 个 5.9 MB 的 .ply」变成**«ply + obj(13.8 MB) + mtl + 图集 PNG(4.67 MB / 2048²)»**。

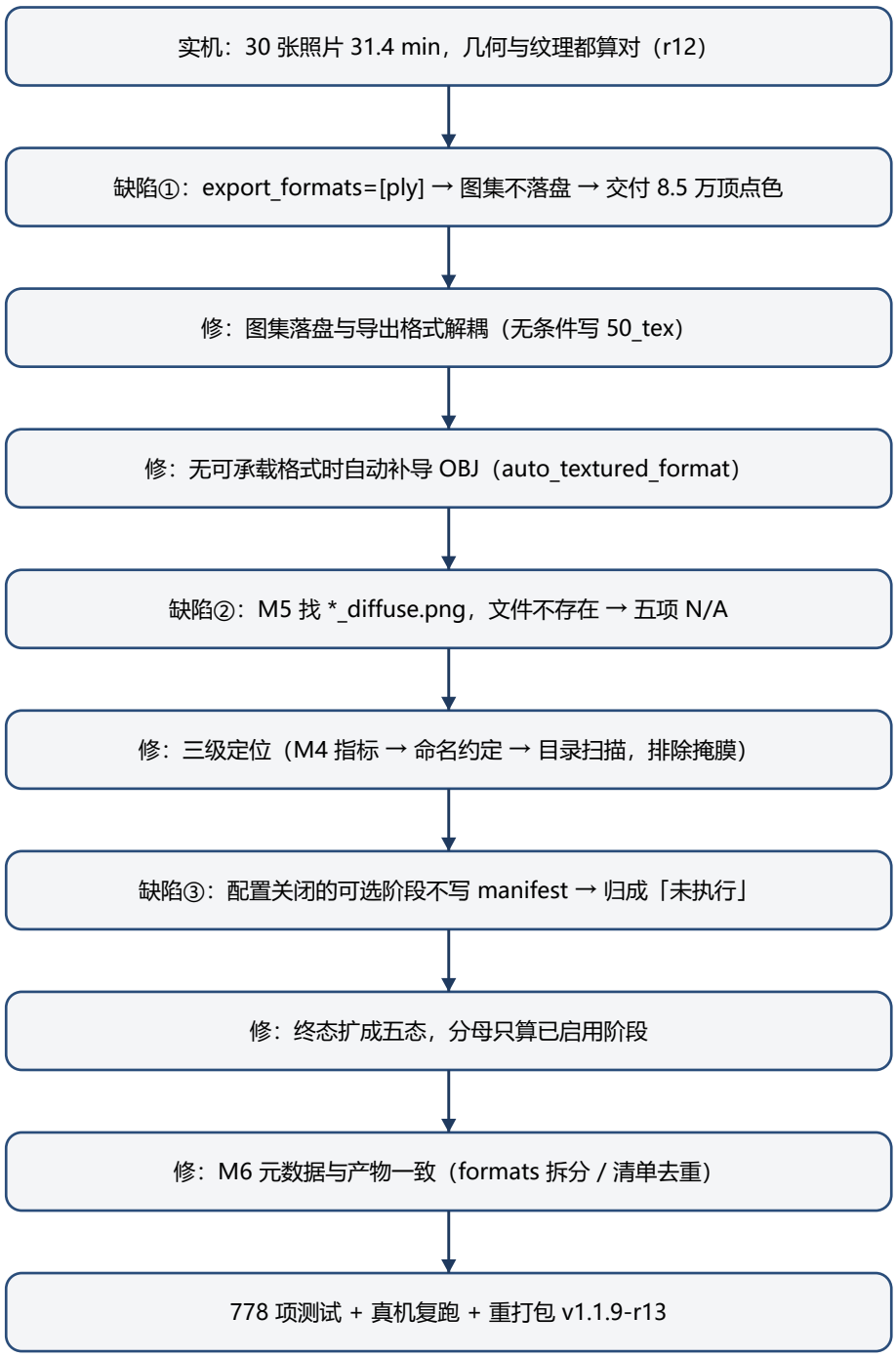


图 1 本版三项缺陷的因果链：① 与 ② 是同一个根因（图集只在导 obj/glb 时才落盘），③ 是报表口径问题，与缓存无关

二、本版改动总表

编号	改动	关键数字 / 影响面
r13-1	新增 _save_atlas_png()，烘焙完**无条件**写图集	50_tex/<item>/<item>_L1.0_diffuse.png，并进 M4 产物列表
r13-2	图集落盘与 export_formats 解耦	只有 ply 也有 PNG；实测 4,674,627 B / 2048²
r13-3	新增 photo_texture.auto_textured_format（默认 true）	无形承载格式时自动补导 OBJ，WARNING + auto_added_formats 留痕
r13-4	M4 指标新增 formats / formats_requested / texture_carrier / atlas_png / atlas_bytes	配置值与实际产物分开记，可自证“配置 ≠ 产物”是不是有意为之
r13-5	新增 publish.deliver_texture_atlas()	图集一律复制进 80_export，无模型引用时 WARNING
r13-6	M6 新增 formats_requested / delivered / missing	缺失格式 WARNING，并指明去看 texture.export_formats
r13-7	M6 把自身 outputs 写进 stage_metrics	修 90_meta 的 delivery.models 恒为空
r13-8	交付清单按 item_id 覆盖而非 append	修 M6 重跑后的「1 件文物 / 2 件已发布」
r13-9	M5 三级定位图集（_locate_texture_atlas）	M4 指标 → 命名约定 → 目录扫描；显式排除 *_mask.png
r13-10	texture_atlas / texture_mask_source 进 manifest	N/A 的成因可审计（无图集 vs 评了没通过）
r13-11	新增 TaskStatus.DISABLED 与 Task.mark_disabled()	五态终态：完成 / 失败 / 跳过 / 未启用 / 未执行
r13-12	编排层把配置关闭阶段写进 manifest（skip_reason 说明哪个开关）	旧实现“不写”，报表只能算成「未执行」
r13-13	报表分母 = 件数 × 阶段数 − 未启用；判定新增「通过（含跳过）」	实测：完成 7 · 未启用 1 · 未执行 0，分母 7 → 通过
r13-14	CSV 状态列中文化，备注列回退 skip_reason	两处口径必须一致，否则用户要对着猜

三、缺陷①：2048² 图集没进交付包

3.1 现象：264 万纹素，只交出 8.5 万

实机项目的配置里写着 texture.export_formats: [ply]。M4 的指标显示：图集 2048×2048、**2,639,926** 个纹素参与采样、岛内填充率 **94.89%**、逐纹素采样耗时 **50.18 s**。而 80_export/兵马俑/ 里只有一个 **85,420 顶点**的 .ply。有效色彩采样从 264 万降到 8.5 万，**差 31 倍**——M4 的主产物 100% 没有交出去。

产物	位置	色彩信息量	能否贴图
<item>_L1.0_diffuse.png	修复前：只在缓存里	2,639,926 纹素	是（UV 图集）
<item>_L1.0.ply	50_tex + 80_export	85,420 顶点色	**否**（PLY 无纹理坐标）
修复后 <item>.obj + .mtl + _diffuse.png	50_tex + 80_export	2,639,926 纹素	是
图集复用缓存	99_runs/atlas_cache/tex/	2,639,926 纹素	是（但不算交付物）

3.2 根因：图集的落盘时机被绑在“导出哪些格式”上

旧实现里，<item>_L1.0_diffuse.png 不是 M4 自己写的，而是 export_textured() 在写 OBJ 时顺手把图集另存为 _diffuse.png 并改写 MTL 的 map_Kd。于是出现一个很隐蔽的耦合：**图集是否存在，取决于用户勾了哪个导出格式。**勾 ply 就只有 ply——而 PLY 格式本身**根本没有放图像纹理的地方。**

```
# 旧实现（简化）
outputs = export_textured(mesh, out_dir, item_id, "L1.0",
                          cfg.photo_texture.export_formats, uv=uv, image=image)
# ↑ formats=[ply] 时：只写 .ply，image 变量用完就丢
#   图集只存在于 cache.put_texture(...) 的缓存副本里
```

这不是“导出失败”，所以没有任何报错。日志里 M4 显示“完成、1 个文件”，报表里 M4 各项纹理指标（填充率 94.89%）也全都在——**指标是算出来的，只是成果没落盘。**

3.3 修复：先把图集写下来，再单独判断有没有模型引用它

修法的核心是把**“产出”与“导出格式”拆成两件事**：图集是 M4 的主产物，烘焙完成就必须落盘；有没有模型能引用它，是另一个独立的问题，用 texture_carrier 单独记录。

```
# 新实现（h3d/photo_texture.py）
requested = [f.lstrip(".").lower() for f in cfg.photo_texture.export_formats]
formats = list(requested)
external = [f for f in formats if f in ("obj", "gltf")]   # 外挂 mtl+png
embedded = "glb" in formats                               # 纹理内嵌
auto_added = []
if not external and not embedded:
    if cfg.photo_texture.auto_textured_format:   # 默认 true
        auto_added = ["obj"]; formats.append("obj"); external = ["obj"]
        log.warning("…没有任何格式能承载图像纹理…已自动补导 OBJ")

atlas_path = os.path.join(out_dir, f"{item_id}_L1.0_diffuse.png")
if "obj" not in formats:          # obj 导出会写同名 PNG，避免重复写盘
    _save_atlas_png(image, atlas_path)
outputs = export_textured(..., formats, uv=uv, image=image)
if not os.path.exists(atlas_path):    # 防御：只有 glb / 导出器改名
    _save_atlas_png(image, atlas_path)
```

auto_textured_format 默认开启的理由：一次 50 秒的采样成果因为一个勾选框变成孤儿文件，代价远大于多导一个 OBJ。它仍**可关**（有些人只要几何存档），关掉时只告警不补导，且指标里 auto_added_formats=[]、texture_carrier=None，审计时能看出是“用户选择不导”而不是“系统漏了”。

3.4 交付层：图集与模型格式无关，一律进包

M6 新增 deliver_texture_atlas()：从 50_tex 找到图集，复制成 80_export/<item>/<item>_diffuse.png。Obj 在场时它与 MTL 的 map_Kd 同名同内容（copy2 幂等）；Obj 不在场时它仍是**唯一可复原纹理的凭据**，同时打 WARNING 说明“交付包只含 ply，无模型可引用图集”。

交付文件（修复前）	交付文件（修复后）	说明
兵马俑.ply 5,886,351 B	兵马俑.ply 5,886,351 B	顶点色，任何软件可开
—	兵马俑.obj 13,805,474 B	带 UV，引用 mtl + 图集
—	兵马俑.mtl 159 B	map_Kd 兵马俑_diffuse.png
—	兵马俑_diffuse.png 4,674,627 B	2048² UV 图集
accuracy.csv / 论文表格	同左	质检报告与论文表格不受影响

四、缺陷②：M5 纹理质检整段失效

4.1 现象：五项 N/A，与 M4 的 94.89% 自相矛盾

M5 的纹理维度有五个字段：texture_available / sharpness / seam / coverage / fill_ratio。实机报表里它们**全部是 N/A / 否**，而同一份报表的「三、纹理质量」小节明明白白写着岛内填充率 **94.89%**。同一张纸上，一边说“贴得很好”，一边说“没评估”。

这是比缺数字更糟的一种状态：**「质检没做」被显示成「质检没有问题」**。如果只扫一眼“纹理达标 = N/A”，很容易读成“不适用”，而不会想到“根本没跑”。

4.2 根因：与缺陷①同一个

M5 只找 <item>_L1.0_diffuse.png。这个文件在缺陷①里已经说明过——**只有导出 obj/glb 时才由导出器写下**。配置是 ply，文件不存在，texture_quality() 直接返回全 None 的 empty 字典。两个缺陷是同一个根因的两个症状。

4.3 修复：三级定位 + 成因分类

```
# h3d/runner/__init__.py
def _locate_texture_atlas(tex_dir, item_id, m4):
    cand = m4.get("atlas_png")          # ① M4 记的绝对路径（权威）
    if cand and os.path.exists(cand):
        return cand
    for name in (f"{item_id}_L1.0_diffuse.png", f"{item_id}_L1_diffuse.png",
                 f"{item_id}_diffuse.png",
                 f"{item_id}_L1.0_0.png", f"{item_id}_L1_0.png"):   # ② 命名约定
        if os.path.exists(os.path.join(tex_dir, name)):
            return os.path.join(tex_dir, name)
    cands = sorted(f for f in os.listdir(tex_dir)                   # ③ 目录扫描
                   if f.endswith(("diffuse.png", "_0.png"))
                   and "mask" not in f.lower())       # 掩膜同尺寸，必须排除
    return os.path.join(tex_dir, cands[0]) if cands else None
```

第 ③ 级为什么要排除 _mask.png：着色掩膜与图集**尺寸完全相同**（都是 2048²），把它当图集读进去，拉普拉斯方差会得到一个没有意义的值，而 texture_available=True 会让报表显示“已评估”——比 N/A 更误导。本版把这条写成了测试断言（test_scan_skips_mask）。

另外加了成因分类：找不到图集时，若 M4 的 paint_mode_effective 是 vertex-color（顶点色链路本来就无图集），打 INFO；若是 photo（照片链路必须有图集却没有），打 WARNING 并指出应检查哪个文件。**“正常”与“异常”必须由系统自己说，不能留给用户猜。**

4.4 实测结果：五项指标回来了，且与 M4 对得上

指标	修复前	修复后（真机）	与 M4 的一致性
texture_available	False	True	—
texture_sharpness	N/A	**71.89**	阈值 3.0，远超
texture_seam	N/A	**2.34**	阈值 100.0，远低于
texture_coverage	N/A	**59.8%**	= M4 coverage_ratio 0.5977

指标	修复前	修复后（真机）	与 M4 的一致性
texture_fill_ratio	N/A	**94.9%**	= M4 island_fill_ratio 0.9489
texture_ok	False	**True**	sharpness / seam / fill 三项全过

五、缺陷③：终态又归并了

5.1 现象：「完成 7 • 未执行 1 → 部分完成」

修复前的报表首屏写着：`阶段统计：完成 7 失败 0 未执行 1`，整体判定「部分完成」。那个「未执行」是 **P1 photo_texture**——它的开关是 `photo_texture.enabled=false`，也就是**用户自己关掉的**。一件跑完全部 7 个已启用阶段的器物，被判成“部分完成”。

更麻烦的是「未执行」这个词在同一张报表里**还承担另一个含义**：上游失败导致该阶段从未到达。两种完全不同的东西共用一个词，用户看到「未执行」时无法判断该去改配置还是去修上游——这正是 1.1.7「跳过≠失败」的同类问题，只是换了个位置复发。

5.2 根因：配置关闭的阶段根本不进 manifest

```
# 旧实现（batch_processor._run_task）
if res.status == TaskStatus.SKIPPED:
    task.mark_skipped(stage_id)
    if _stage_enabled(stage_id, cfg):
        log.info("○ %s 跳过：%s（%s）", stage_id, task.item_id, reason)
    else:
        log.debug("可选 stage 未启用，跳过：%s", key)  # ← 只有 debug，不写 manifest
    continue
```

报表是**纯 manifest 视图**。`manifest.stages` 里没有 `兵马俑/P1` 这个键，报表只能走“缺失阶段”分支，于是它和“上游失败”一起被计入 `n_pending`。也就是说：**数据侧根本没记录“这是用户关掉的”，报表侧无从区分。**

5.3 修复：阶段终态扩成五态

终态	manifest status	含义	计入分母
完成	`done`	本次跑完	是
失败	`fail`	本次失败，下游未执行	是
跳过	`skipped` + `skip_kind`	已启用但没跑：条件不满足 / 降级 / 续跑复用	是
未启用	`disabled`	配置里关掉了（`*.enabled=false`）	**否**
未执行	（键不存在）	本次运行无该阶段记录（多为上游失败）	是

「未启用」与「跳过」的区分不只是一个词：**前者是用户的意图，后者是运行时的意外。**所以 `Task.mark_disabled()` **不修改任务级状态**——一件器物不该因为一个从未参与的可选阶段变成 SKIPPED；而 `mark_skipped()` 保持原行为（整件都没跑时统计里要看得见）。

```
# 新实现：配置关闭 -> disabled + 写明是哪个开关
task.mark_disabled(stage_id)
manifest.record(key, TaskStatus.DISABLED.value, [], None,
                skip_reason=_disabled_reason(stage_id, cfg))

# 已启用但条件不满足 -> skipped + skip_kind
task.mark_skipped(stage_id)
manifest.record(key, TaskStatus.SKIPPED.value, [], None,
                skip_kind="condition", skip_reason=reason[:300])
```

开关	写进 skip_reason 的内容
sfm_solve.enabled=false	配置关闭：sfm_solve.enabled=false（未启用照片位姿解算）
photo_texture.enabled=false **且** M4 已用 photo 模式	**配置关闭：M4 已按 paint_mode=photo 输出照片纹理，P1 与 M4 互斥（同目录重复投影）**
photo_texture.enabled=false	配置关闭：photo_texture.enabled=false（可选增强未启用）
runner.x1_enabled=false	配置关闭：runner.x1_enabled=false（未启用多时相形变检测）

5.4 报表：分母只算已启用阶段

```
分母 = 件数 × 阶段数 − 未启用     # 例：1 × 8 − 1 = 7
判定：
  有失败        -> 失败
  无失败、无未执行、无跳过 -> 通过
  无失败、无未执行、有跳过 -> 通过（含跳过）
  无失败、有未执行          -> 部分完成
```

实测同一项目（只重跑 M4/M5/M6，S1~M3 断点续跑）修复后的首屏：

```
阶段统计：完成 7   失败 0   跳过 0   未启用 1   未执行 0
分母 7 = 1 件 × 8 阶段 − 未启用 1（配置关闭的阶段不计入评估范围）
整体判定：通过   已启用的 7 个阶段全部完成（另有 1 个阶段被配置关闭，
与本次结论无关）。

兵马俑 | P1 | 未启用 | — | 配置关闭：M4 已按 paint_mode=photo 输出照片纹理，
                          P1 与 M4 互斥（同目录重复投影）
```

顺带修了两个副产物：① 跳过/未启用的 `elapsed_s` 改记 `None`，报表显示「—」而不是「0.00」（后者会被读成“跑了但瞬间结束”）；② CSV 的状态列同步中文化（`完成/失败/跳过·条件不满足/未启用/未执行`），备注列在 `reason` 缺省时回退 `skip_reason`——旧实现只读 `reason`，于是“为什么没跑”在 CSV 里彻底丢失。

六、缺陷④：元数据与产物对不上

6.1 三处不一致

位置	旧值	实际产物	根因
90_meta/<item>_meta.json → delivery.models	[]（空列表）	有 1~2 个模型文件	build_meta 取 stage_metrics['M6']['outputs']，而 M6 只被 setdefault 成 {'status': 'done'}
manifest <item>/M6 → formats	[ply, obj, glb]	只有 ply	记的是配置请求值，没有记实际交付值
90_meta/delivery_manifest.json	2 件 / 2 已发布（实际 1 件）	1 件	M6 重跑时无脑 append，不去重

6.2 修复

① M6 把自己的 outputs 写进 stage_metrics["M6"]；② formats 拆成 formats_requested / formats_delivered / formats_missing 三项，缺失时 WARNING 并指明“多为 M4 未导出该格式，见 texture.export_formats”；③ 交付清单按 item_id 覆盖。修复后的 90_meta：

```
"delivery": {
  "models": ["...\\兵马俑.obj", "...\\兵马俑.ply", "...\\兵马俑_diffuse.png"],
  "qc": "done"
}
```

报表「五、交付包索引」也扩成 7 列：文物编号 / 状态 / 交付模型数 / **实际格式** / **缺失格式** / **纹理图集** / **纹理承载**。最后两列是本版的关键——前者证明图集进了包，后者说明有没有模型真正引用它（无 时包里的图集是孤立文件）。实测该行：ply、obj | glb | 兵马俑_diffuse.png | obj。

七、验证

7.1 单元测试

测试文件	项数	锁住的行为
test_texture/test_atlas_delivery.py	11	图集落盘与格式解耦 / 自动补导可开可关 / 三级定位排除掩膜 / M6 复制图集 / M5 五项指标非 N/A
test_report/test_terminal_states.py	10	mark_disabled 不动任务状态 / 五态在 manifest 里可区分 / 分母扣掉未启用 / 判定三档 / 失败优先
全量	**778**	1.1.8 为 757，本版 +21

7.2 真机复跑（同一 30 张照片项目）

只删掉 manifest 里的 M4 条目后重跑，S1~M3 断点续跑复用产物，M4 命中图集缓存（**2 s**，对比冷启动 83.6 s），M5/M6 重算。关键日志：

```
[INFO] UV 图集复用：兵马俑（几何指纹 4b795526747e，跳过 xatlas 展开）
[INFO] 纹理图集复用：兵马俑（跳过逐纹素采样与融合）
[WARNING] 照片纹理已烘焙 2048² 图集，但 export_formats=['ply'] 里没有任何格式
        能承载图像纹理——已自动补导 OBJ（auto_textured_format=true）
[INFO] M4 完成：兵马俑（paint_mode=photo，照片投影纹理，3 文件）
[INFO] M5 纹理质量：sharpness=71.89 seam=2.34 填充率=94.9% 覆盖率=59.8%
[INFO] M5 完成：兵马俑（coverage=82.8%, rmse=0.0426 模型单位 -> pass）
[INFO] 交付包贴图随附：兵马俑（材质 兵马俑.mtl，贴图 ['兵马俑_diffuse.png']）
[WARNING] 交付格式缺失：['glb']（请求 ['ply','obj','glb']，实际 ['ply','obj']）
[INFO] 交付包图集：兵马俑_diffuse.png（由 OBJ 材质引用）
[INFO] 交付清单：….json（1 件，1 已发布）
```

验收项	修复前	修复后
交付包文件数	3（ply + 2 csv）	**7**（ply + obj + mtl + 图集 + 3 csv）
图集是否在交付包	否	**是**（4,674,627 B / 2048²）
纹理承载	—	**obj**（mtl map_Kd 引用）
M5 纹理指标	5 项全 N/A	**5 项全部算出**，填充率 94.9% 与 M4 一致
报表首屏判定	部分完成（未执行 1）	**通过**（未启用 1，分母 7）
delivery.models	[]	3 项（obj / ply / png）
交付清单条数	2（重复登记）	**1**

八、已知限制与下一步

1) **图集仍可能与模型格式不匹配（可配置）**：把 auto_textured_format 关掉后，交付包会拿到一个孤儿图集。这是刻意保留的（有人只要几何存档），但需要在验收时看报表「纹理承载」列——显示 无 就说明图集没被引用。

2) **交付格式缺失只告警不补导**：publish.formats 与 texture.export_formats 是两处独立配置，前者要 glb 而后者没导时，本版只在日志与报表标出，不会替用户改配置。理由是“导出哪些格式”属于交付决策，不该由系统代劳；但这一条会在验收时反复出现，下一步考虑在 GUI 里把两个勾选框联动。

3) **X1/X2 不进报表状态表**：manifest 里它们现在也会记 disabled（所以 metrics_summary.csv 有 10 行），但 HTML 状态表只渲染 8 个阶段（X1 有独立的「多时相形变」小节）。分母按 8 算，两者口径不同但各自自洽，后续若要统一，需先决定 X1/X2 是否算“主线阶段”。

4) **M4 图集落盘增加了约 0.2 s**（2048^2 PNG 编码）。实测该件 export_s=0.22 → 0.4 s 量级，相对 50 s 的采样可忽略；但若图集升到 4096^2，建议改成“先写 PNG 再让 OBJ 直接引用它”，避免导出器重复编码一次。

5) **真机待验**：本版只在单件纯照片项目上复跑过。多件批处理下「未启用 vs 跳过」的计数、以及 workers>1 时新增的 skipped/disabled 落盘路径尚未实测（并行分支已按同一语义改造并有单测覆盖）。

> **本版可复用的一条工程经验**：**“算出来了”和“交出去了”是两件事。**M4 的 94.89% 填充率是真实算出来的，日志也报了“完成”，但成果留在缓存目录里——于是交付包里没有它，质检也找不到它。凡是**耗时很长的中间产物**，它的落盘时机不能挂在“下游要不要用”这个条件上；而凡是**报表里出现 N/A 的地方**，都必须能回答“是没数据还是没做”——否则 N/A 就变成了最危险的一种“通过”。